

Predicting Financial System Vulnerability in Finland Using Macroeconomic Indicators

1. Introduction:

Background:

In the evolving field of economics, there exists a continual need to assess the vulnerabilities of different sectors, countries, or commodities. Understanding these vulnerabilities aids in making informed decisions for businesses, policymakers, and investors. The application discussed here pertains to predicting economic vulnerability of a country or another entity based on specific economic indicators. Leveraging machine learning, we seek to forecast the "Vulnerability Score" of Finland based on economic data.

Outline:

Section 2) **Problem Formulation:** This section defines the problem, and discusses data points, features, labels, and dataset sources.

Section 3) **Methods:** Here, we detail data preprocessing, feature selection, the choice of ML models (Linear Regression and Random Forest Regressor), loss functions, and model validation approaches.

Section 4) **Results:** We present training, validation, and testing errors for both models across the different features, discussing model trade-offs and explaining our choice of the final method.

Section 5) **Conclusion:** The last part summarizes findings, highlights challenges, and suggests future directions on how to improve the project.

2. Problem Formulation:

ML Problem Definition:

Our task is to predict a continuous "Vulnerability Score" of Finland, calculated from certain economic indicators. This is a supervised regression problem.

Data Points, Features, and Labels:

- **Data Points:** Each data point represents the value of an economic indicator (which are listed in the features part) at a specific point in time, in this case every month from January 1980 up to 2022. The data is continuous. All the economic data is concerning Finland only.
- **Features:** We are using multiple features such as the Consumer Price Index (CPI), Gross Domestic Product (GDP), and Consumer Prices (called "Prices" in the code). We are also using a "weighted feature", which is a combination of all the above features, with their own weights (CPI 40%, GDP 30% and Consumer Prices 30%). All the features are using the "rolling window" (or "rolling mean") technique, which in practice means that the feature is the mean of the values of the past 3 months.

- **Labels:** The label is the "Vulnerability Score", which is a continuous variable indicating the level of economic vulnerability. The vulnerability score is calculated using the CPI, GDP and Consumer Prices, but it is not well defined and can be modified to also consider some other economic indicators (e.g. Unemployment rate).

Source of Dataset:

The data for GDP and Consumer Prices is from Federal Reserve Economic Data (FRED) and the data for CPI is from Statistics Finland.

3. Methods:

Dataset and Preprocessing:

We are using 3 datasets (one for each feature) from Federal Reserve Economic Data (FRED) and Statistics Finland, which in total have $504 \times 3 = 1512$ data points. The datasets are normalized values of the corresponding economic indicator for each month. Some data preprocessing was needed. Mainly making all the data start from the same month and handling all of the NaN values in the data. Also the Vulnerability Rate needed to be calculated from the data, adding another 504 data points to be used as the label.

Feature Selection Process:

Feature selection was influenced by prior knowledge of the subject and preliminary data visualization. The core features (CPI, GDP, and Consumer Prices) were identified as crucial indicators in economic analyses. We considered some other features such as CLI or Unemployment Rate, but they were discarded when preliminarily visualizing the data. Additionally, we synthesized a "Weighted Feature" by integrating multiple features with specific weights to encapsulate more nuanced relationships. All features are modified to use the rolling window technique, as explained in the "Features" section above, to see the lagged effects of the features on the label and to be able to forecast into the future.

If we were to continue the feature selection further, we would use the results of the Random Forest Regressor's feature importances.

Choice of ML Models:

Two primary models were explored:

1. **Linear Regression model:** While our data may have complex relationships, it's beneficial to have a linear baseline model for comparison. The Linear predictor maps can show a linear relationship between the features and the label.
2. **Random Forest Regressor model:** Chosen due to its ability to handle complex relationships between variables, cater to both linear and non-linear patterns, and provide feature importances. It typically works well without much fine-tuning.

Loss Functions:

- For **Linear Regression**, the inherent Mean Squared Error (MSE) was used since it penalizes larger errors more severely, which is suitable for our regression problem.
- For **Random Forest Regressor**, MSE was also adopted as the loss function to maintain consistency in evaluation metrics.

- For the validation and training errors we are using the average MSE from all 5 folds, since we are using k-fold cross-validation.

Model Validation:

The data was split into training and test sets using a 80:20 ratio. For validation we are using k-fold cross-validation with 5 folds. The reason behind this choice was to ensure that, even with our limited data, the model has a sufficiently large training set to capture underlying patterns, while also reserving enough data for testing to evaluate and ensure model generalizability.

4. Results:

Comparison of the methods:

TR = average training error across all folds, VA = average validation error across all folds, TE = testing error

Errors:	CPI	GDP	Consumer Prices	Weighted Feature
Linear Regression TR	0.0047	0.0101	0.0134	0.0009
Random Forest TR	0.0018	0.0018	0.0026	0.0002
Linear Regression VA	0.0048	0.0101	0.0136	0.0009
Random Forest VA	0.0051	0.0129	0.0185	0.0013
Linear Regression TE	0.0053	0.0096	0.0122	0.001
Random Forest TE	0.0064	0.0100	0.0152	0.0015

For each feature and ML method, the training errors were consistently lower than the validation errors. This indicates that the models fit the training data slightly better than unseen validation data.

When comparing the two algorithms, Random Forest tends to have lower training errors than Linear Regression for each feature. However, Random Forest has higher validation errors for some features compared to Linear Regression, indicating very possible overfitting.

Choosing final method:

Given the results, the final chosen method is Linear Regression trained on the Weighted Feature. This decision is based on having the lowest average validation MSE of 0.0009 and the lowest test MSE of 0.001 among all combinations. With some fine tuning and by addressing the potential overfitting problem of the Random Forest with Weighted Feature, it could probably surpass the Linear Regression.

Test Error:

The test set was constructed by splitting the original dataset into 80% training/validation (Using k-fold) and 20% testing. This split ensures that the model can be evaluated on unseen data, while leaving enough data for training the model.

For the chosen final method (Linear Regression on Weighted Feature), the test error is 0.001.

5. Conclusion:

In this report, we applied machine learning to forecast Finland's economic vulnerability using key indicators like CPI, GDP, and Consumer Prices. The data was sourced from Federal Reserve Economic Data (FRED) and Statistics Finland and was analyzed using two primary models: Linear Regression and Random Forest Regressor.

Our findings indicate that while our models fit the training data well, they were slightly less accurate with the validation data. Among the two models, the Random Forest showcased lower training errors but also indicated potential overfitting in certain scenarios. On the other hand, the Linear Regression model, especially when using the Weighted Feature, gave the most consistent results across training, validation, and test datasets.

While the results with Linear Regression are promising, there's room for enhancement. The slightly higher validation errors in the Random Forest model hint at possible improvements, especially if overfitting can be addressed.

There were a few challenges with our approach:

1. **Data Depth:** Our dataset could benefit from more data points to improve model accuracy. Finding extensive economic data proved to be more difficult than we first imagined.
2. **Features Used:** The weighted feature was helpful, but adding more indicators might refine the predictions further. Potential features like the unemployment rate could be re-investigated.
3. **Model Tuning:** The Random Forest model might require some adjustments, possibly in terms of hyperparameters, to reduce overfitting.

For future directions, we could consider:

- Expanding the feature set with more economic indicators.
- Trying out other regression models or techniques.
- Using different loss functions for more aligned results.
- Gathering newer data.
- Using a more well defined Vulnerability Score to better reflect the real world.

In summary, our current approach provides valuable insights into predicting Finland's economic state. However, there's much potential for further refinement.

Appendix:

```
%config Completer.use_jedi = False
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

from sklearn.preprocessing import PolynomialFeatures
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import MinMaxScaler

from sklearn.ensemble import RandomForestRegressor

from sklearn.model_selection import cross_val_score, train_test_split, KFold
```

```
#Data Loading and Preprocessing:

#CPI:
dfCPI = pd.read_csv('CPI_monthly.csv', skiprows=1)

new_columns_CPI = ['Month', 'CPI']
dfCPI.columns = new_columns_CPI

dfCPI['Month'] = dfCPI['Month'].str.replace(r'^(\d{4})-(\d{2})-(\d{2})$', '□',
    ↪r'\1M\2', regex=True)

dfCPI = dfCPI.reset_index(drop=True)

#GDP:
dfGDP = pd.read_csv('GDP_monthly.csv')

new_columns_GDP = ['Month', 'GDP']
```

```

dfGDP.columns = new_columns_GDP

dfGDP['Month'] = dfGDP['Month'].str.replace(r'^(\d{4})-(\d{2})-(\d{2})$', 
    ↪r'\1M\2', regex=True)
dfGDP = dfGDP[dfGDP['Month'] >= '1980M01']

dfGDP = dfGDP.reset_index(drop=True)

#Consumer Prices:
dfC = pd.read_csv('Consumer_Prices.csv')

new_columns_C = ['Month', 'Prices']
dfC.columns = new_columns_C

dfC['Month'] = dfC['Month'].str.replace(r'^(\d{4})-(\d{2})-(\d{2})$', r'\1M\2', 
    ↪regex=True)
dfC = dfC[dfC['Month'] >= '1980M01']

dfC = dfC.reset_index(drop=True)

# Merge all dataframes on the 'Month' column
dfAll = pd.merge(dfC, dfCPI, on='Month', how='inner')
dfAll = pd.merge(dfAll, dfGDP, on='Month', how='inner')

# Normalize the data
scaler = MinMaxScaler()
dfAll[['CPI', 'GDP', 'Prices']] = scaler.fit_transform(dfAll[['CPI', 'GDP', 
    ↪'Prices']])

# Calculate vulnerability score
dfAll['Vulnerability_Score'] = 0.4 * dfAll['CPI'] + 0.3 * dfAll['GDP'] + 0.3 * 
    ↪dfAll['Prices']

# Creating the weighted feature
dfAll['Weighted_Feature'] = 0.4 * dfAll['CPI'] + 0.3 * dfAll['GDP'] + 0.3 * 
    ↪dfAll['Prices']

#Splitting data into training and test sets and configuring the k-fold
    ↪cross-validation:

# Setting window size for rolling mean
window_size = 3

```

```

# Rolling mean with shift for future prediction
for feature in ['CPI', 'GDP', 'Prices', 'Weighted_Feature']:
    dfAll[f'{feature}_rolling'] = dfAll[feature].rolling(window=window_size).
    ↪mean().shift(1)

# Split the data into training and test sets (80% train and 20% test)
train_data, test_data = train_test_split(dfAll, test_size=0.20, random_state=42)

# Define the KFold cross-validation
kf = KFold(n_splits=5, shuffle=True, random_state=42)

```

#Using Linear Regression:

```

#For-loop to go through all the features
for feature in ['CPI', 'GDP', 'Prices', 'Weighted_Feature']:
    mse_values = [] # store validation MSE values for each fold
    training_mse_values = [] # store training MSE values for each fold

    for train_index, val_index in kf.split(train_data):
        train_fold, val_fold = train_data.iloc[train_index], train_data.
        ↪iloc[val_index]

        # Filter out rows with NaN values
        df_train = train_fold.dropna(subset=[f'{feature}_rolling',
        ↪'Vulnerability_Score'])
        df_val = val_fold.dropna(subset=[f'{feature}_rolling',
        ↪'Vulnerability_Score'])

        X_train = df_train[[f'{feature}_rolling']]
        y_train = df_train['Vulnerability_Score']

        X_val = df_val[[f'{feature}_rolling']]
        y_val = df_val['Vulnerability_Score']

        # Train linear regression
        regr = LinearRegression().fit(X_train, y_train)

        # Calculate training MSE
        training_predictions = regr.predict(X_train)
        training_mse_values.append(mean_squared_error(y_train,
        ↪training_predictions))

        # Calculate validation MSE

```

```

        val_predictions = regr.predict(X_val)
        mse_values.append(mean_squared_error(y_val, val_predictions))

# Average training and validation MSE across all folds
        average_training_mse = np.mean(training_mse_values)
        average_val_mse = np.mean(mse_values)

        print(f"Training MSE for {feature} using Linear Regression:␣
↪{average_training_mse}")
        print(f"Validation MSE for {feature} using Linear Regression:␣
↪{average_val_mse}")

# Train the model on the entire training set and evaluate on the test set
        df_train_all = train_data.dropna(subset=[f'{feature}_rolling',␣
↪'Vulnerability_Score'])
        df_test = test_data.dropna(subset=[f'{feature}_rolling',␣
↪'Vulnerability_Score'])

        X_train_all = df_train_all[[f'{feature}_rolling']]
        y_train_all = df_train_all['Vulnerability_Score']

        X_test = df_test[[f'{feature}_rolling']]
        y_test = df_test['Vulnerability_Score']

        final_model = LinearRegression().fit(X_train_all, y_train_all)
        test_predictions = final_model.predict(X_test)

        test_mse = mean_squared_error(y_test, test_predictions)
        print(f"Testing MSE for {feature} using Linear Regression: {test_mse}\n")

```

Training MSE for CPI using Linear Regression: 0.0047131113442739295
 Validation MSE for CPI using Linear Regression: 0.004792190901192275
 Testing MSE for CPI using Linear Regression: 0.005257721253758835

Training MSE for GDP using Linear Regression: 0.010083875533927806
 Validation MSE for GDP using Linear Regression: 0.010174187826180655
 Testing MSE for GDP using Linear Regression: 0.009576483603510308

Training MSE for Prices using Linear Regression: 0.013437040666816374
 Validation MSE for Prices using Linear Regression: 0.013608173242493313
 Testing MSE for Prices using Linear Regression: 0.012181937695801373

Training MSE for Weighted_Feature using Linear Regression: 0.000892141419772252
 Validation MSE for Weighted_Feature using Linear Regression:
 0.0009076159973588742

Testing MSE for Weighted_Feature using Linear Regression: 0.0009911021817979003

```
#Using Random Forest regressor:

#For-loop to go through all the features
for feature in ['CPI', 'GDP', 'Prices', 'Weighted_Feature']:
    mse_values_rf = [] # to store validation MSE values for each fold for
    ↪Random Forest
    training_mse_values_rf = [] # to store training MSE values for each fold
    ↪for Random Forest

    for train_index, val_index in kf.split(train_data):
        train_fold, val_fold = train_data.iloc[train_index], train_data.
        ↪iloc[val_index]

        # Filter out rows with NaN values
        df_train = train_fold.dropna(subset=[f'{feature}_rolling',
        ↪'Vulnerability_Score'])
        df_val = val_fold.dropna(subset=[f'{feature}_rolling',
        ↪'Vulnerability_Score'])

        X_train = df_train[[f'{feature}_rolling']]
        y_train = df_train['Vulnerability_Score']

        X_val = df_val[[f'{feature}_rolling']]
        y_val = df_val['Vulnerability_Score']

        # Train Random Forest
        rf = RandomForestRegressor(n_estimators=100, random_state=42).
        ↪fit(X_train, y_train)

        # Calculate training MSE for this fold and append to list for Random
        ↪Forest
        training_predictions_rf = rf.predict(X_train)
        training_mse_values_rf.append(mean_squared_error(y_train,
        ↪training_predictions_rf))

        # Calculate validation MSE for this fold and append to list for Random
        ↪Forest
        val_predictions_rf = rf.predict(X_val)
        mse_values_rf.append(mean_squared_error(y_val, val_predictions_rf))

        # Compute average training and validation MSE across all folds for Random
        ↪Forest
        average_training_mse_rf = np.mean(training_mse_values_rf)
        average_val_mse_rf = np.mean(mse_values_rf)
```

```

    print(f"Training MSE for {feature} using Random Forest:␣
↪{average_training_mse_rf}")
    print(f"Validation MSE for {feature} using Random Forest:␣
↪{average_val_mse_rf}")

    # Train the model on the entire training set and evaluate on the test set
    df_train_all = train_data.dropna(subset=[f'{feature}_rolling',␣
↪'Vulnerability_Score'])
    df_test = test_data.dropna(subset=[f'{feature}_rolling',␣
↪'Vulnerability_Score'])

    X_train_all = df_train_all[[f'{feature}_rolling']]
    y_train_all = df_train_all['Vulnerability_Score']

    X_test = df_test[[f'{feature}_rolling']]
    y_test = df_test['Vulnerability_Score']

    final_model_rf = RandomForestRegressor(n_estimators=100, random_state=42).
↪fit(X_train_all, y_train_all)
    test_predictions_rf = final_model_rf.predict(X_test)

    test_mse_rf = mean_squared_error(y_test, test_predictions_rf)
    print(f"Testing MSE for {feature} using Random Forest: {test_mse_rf}\n")

```

Training MSE for CPI using Random Forest: 0.0017694873678806855
 Validation MSE for CPI using Random Forest: 0.005132561283406439
 Testing MSE for CPI using Random Forest: 0.00642598299848368

Training MSE for GDP using Random Forest: 0.0018444465353047376
 Validation MSE for GDP using Random Forest: 0.012867098323281473
 Testing MSE for GDP using Random Forest: 0.01002953677625514

Training MSE for Prices using Random Forest: 0.0026219790263394137
 Validation MSE for Prices using Random Forest: 0.018494850423228175
 Testing MSE for Prices using Random Forest: 0.015217960168261603

Training MSE for Weighted_Feature using Random Forest: 0.0001731798045196146
 Validation MSE for Weighted_Feature using Random Forest: 0.001322664078204153
 Testing MSE for Weighted_Feature using Random Forest: 0.001456257205603778

```

#Using Random Forest with all features to get the Feature Importances:

# Rolling mean with shift for future prediction
features = ['CPI', 'GDP', 'Prices']
for feature in features:

```

```

    dfAll[f'{feature}_rolling'] = dfAll[feature].rolling(window=window_size).
    ↪mean().shift(1)

# Training Random Forest with all features
X_train_all = train_data.dropna(subset=[f'{feature}_rolling' for feature in
    ↪features] + ['Vulnerability_Score'])
y_train_all = X_train_all['Vulnerability_Score']

# Using rolling values for all features
X_train_all = X_train_all[[f'{feature}_rolling' for feature in features]]

# Test set
df_test = test_data.dropna(subset=[f'{feature}_rolling' for feature in
    ↪features] + ['Vulnerability_Score'])
X_test = df_test[[f'{feature}_rolling' for feature in features]]
y_test = df_test['Vulnerability_Score']

# Train the Random Forest model
rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train_all, y_train_all)

# Predict on the test set
test_predictions = rf.predict(X_test)
mse_rf = mean_squared_error(y_test, test_predictions)

print("\nFeature Importances:")
for feature, importance in zip(features, rf.feature_importances_):
    print(f"Importance of {feature}: {importance}")

```

Feature Importances:
 Importance of CPI: 0.789031704201618
 Importance of GDP: 0.1504525722341656
 Importance of Prices: 0.06051572356421643

```

#Plotting all the different features using the two methods:

# Store models and their predictions in a dictionary for easy plotting
models = {
    "Linear Regression": {
        "model": LinearRegression(),
        "predictions": None
    },
    "Random Forest": {
        "model": RandomForestRegressor(n_estimators=100, random_state=42),
        "predictions": None
    }
}

```

```

}

# Plot predictions against true values for each feature and model
for feature in ['CPI', 'GDP', 'Prices', 'Weighted_Feature']:

    df_train_all = train_data.dropna(subset=[f'{feature}_rolling',
↪'Vulnerability_Score'])
    df_test = test_data.dropna(subset=[f'{feature}_rolling',
↪'Vulnerability_Score'])

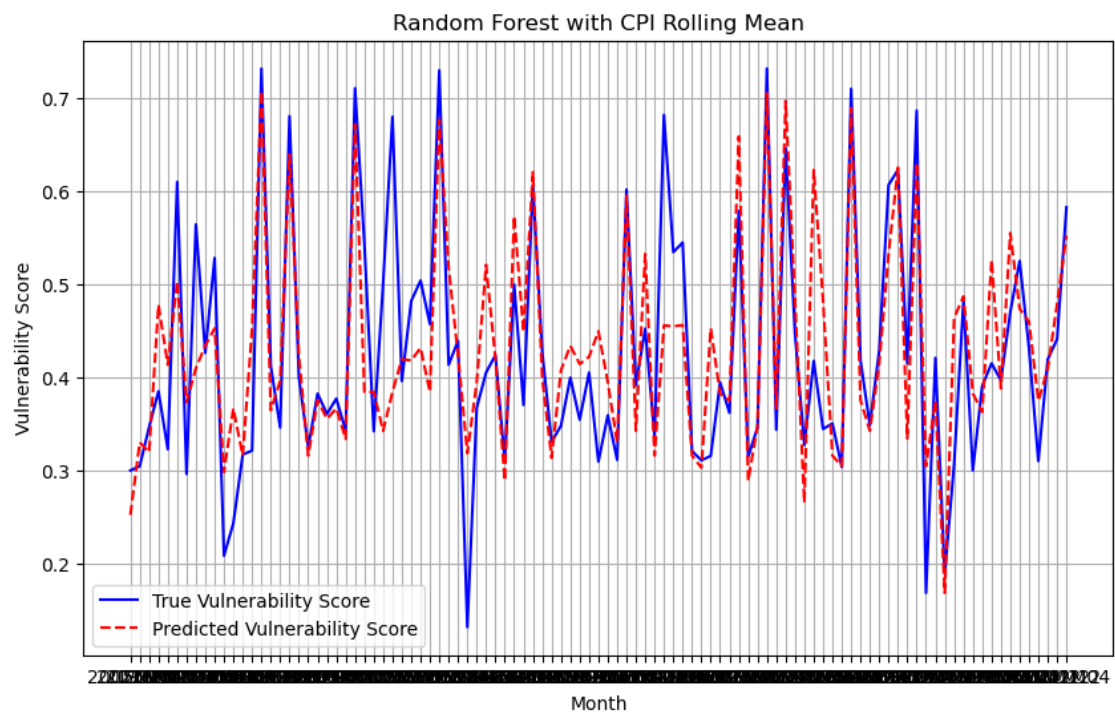
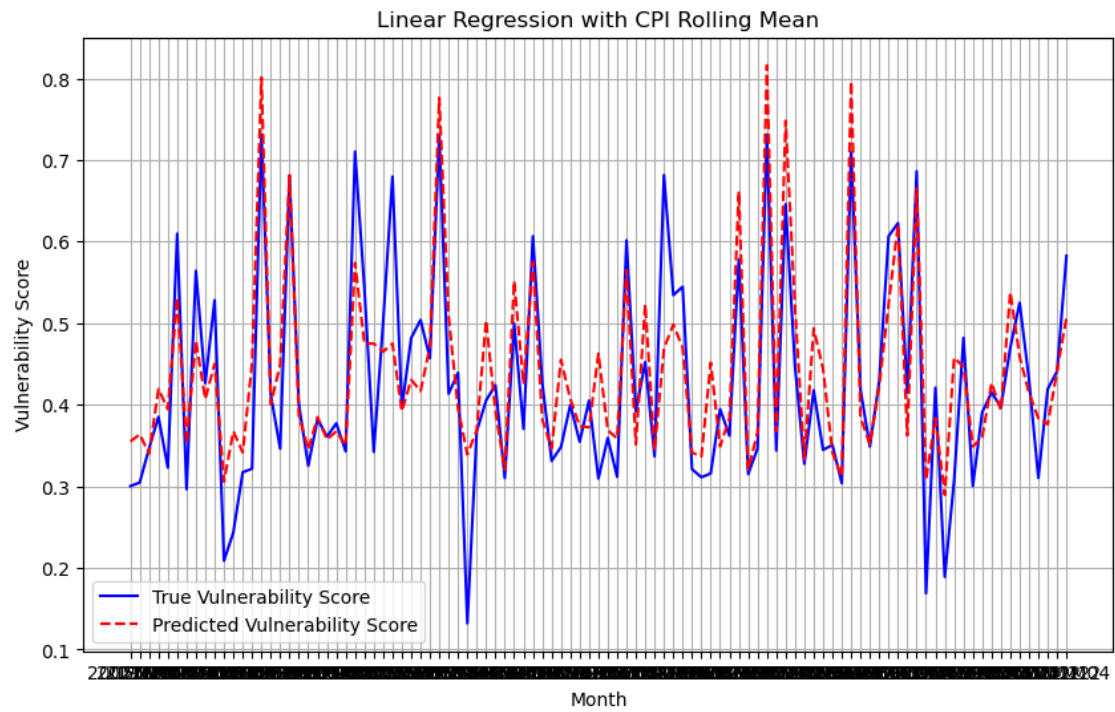
    X_train_all = df_train_all[[f'{feature}_rolling']]
    y_train_all = df_train_all['Vulnerability_Score']

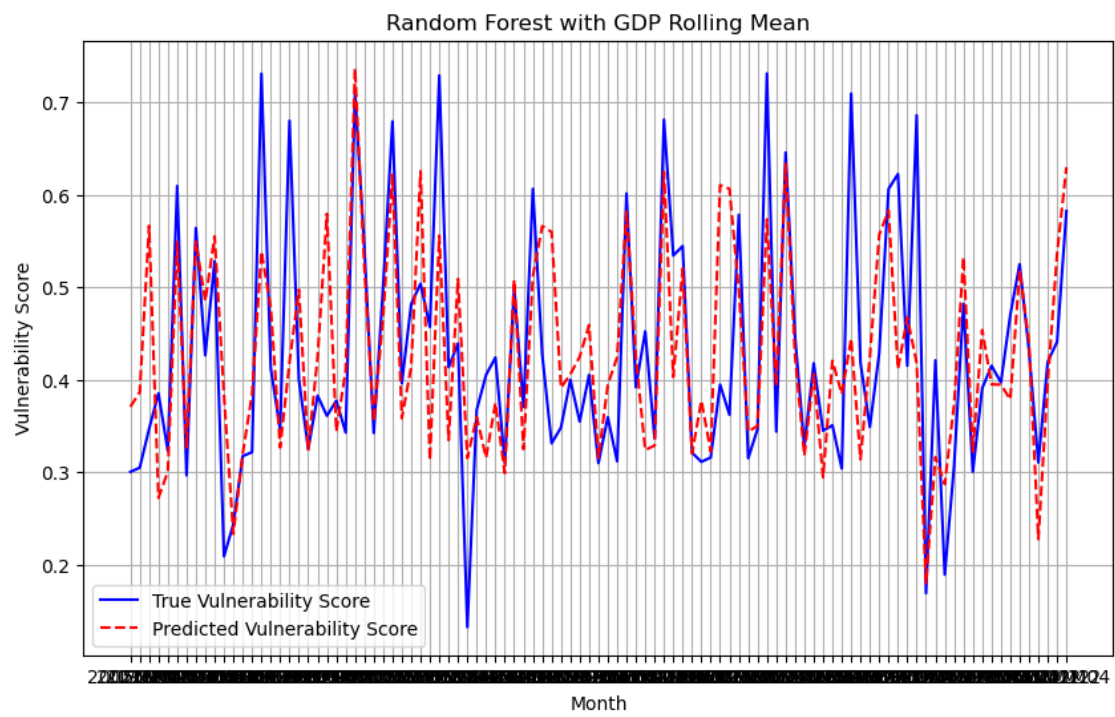
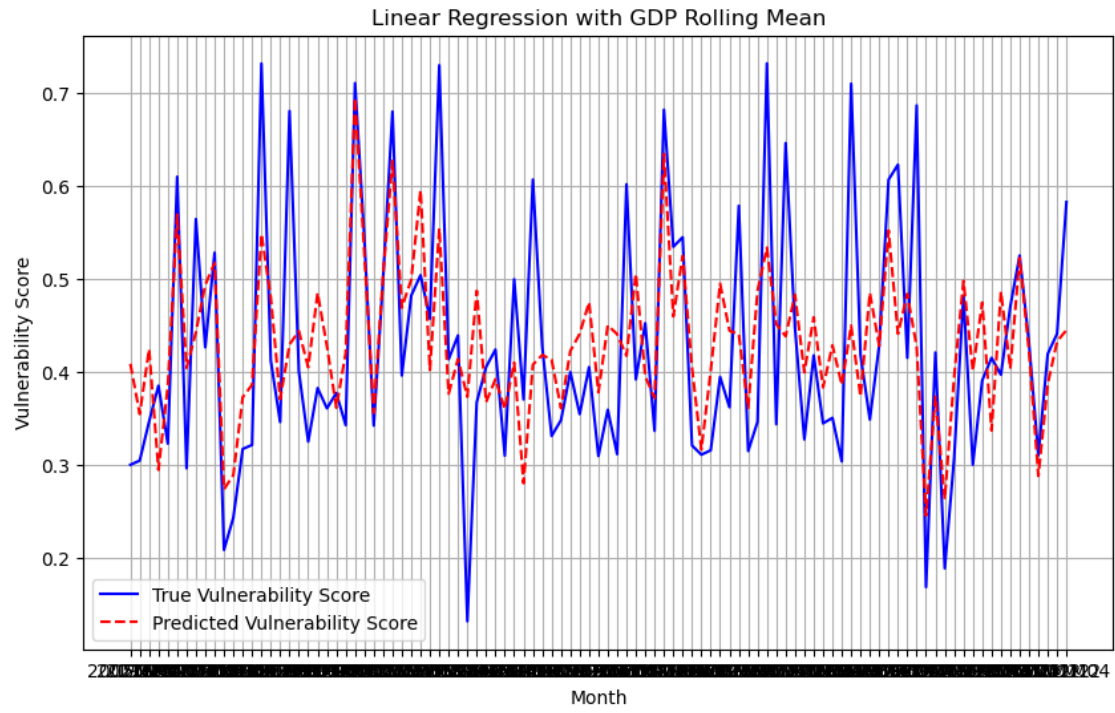
    X_test = df_test[[f'{feature}_rolling']]
    y_test = df_test['Vulnerability_Score']

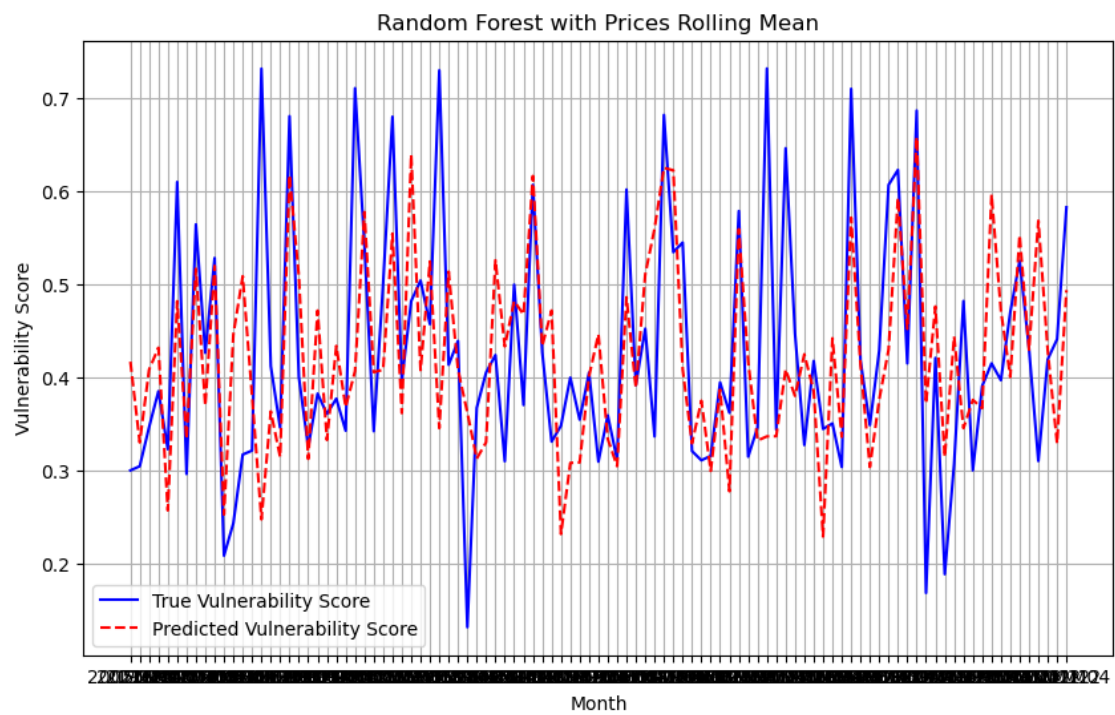
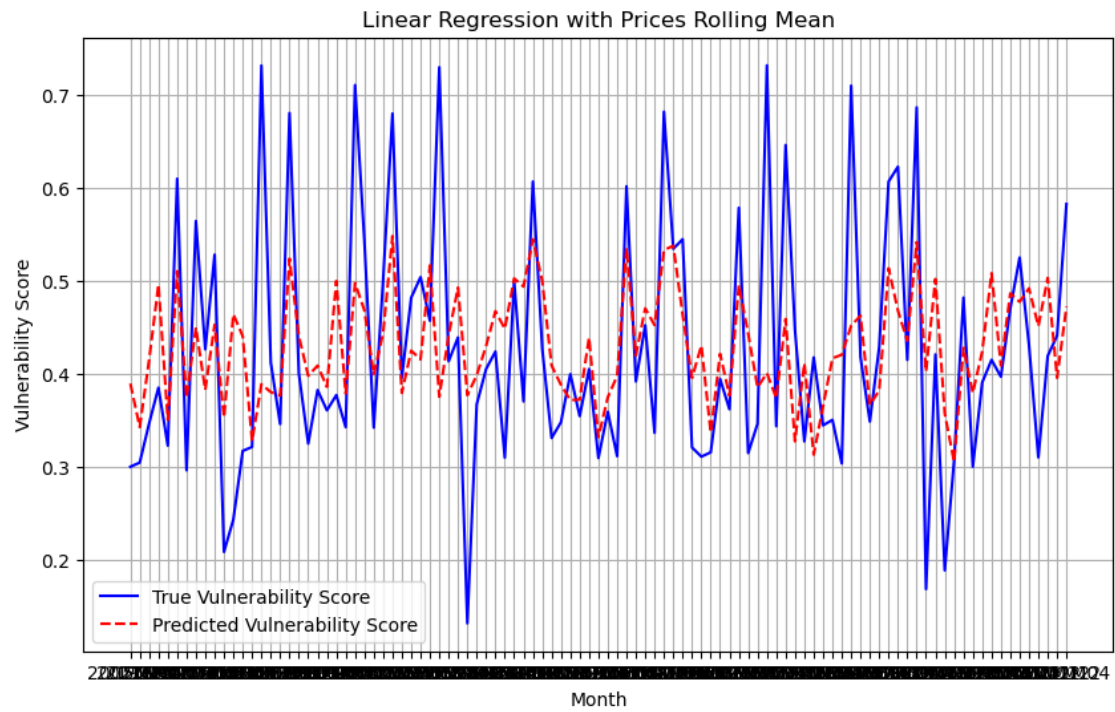
    for model_name, model_info in models.items():
        model = model_info["model"]
        model.fit(X_train_all, y_train_all)
        model_info["predictions"] = model.predict(X_test)

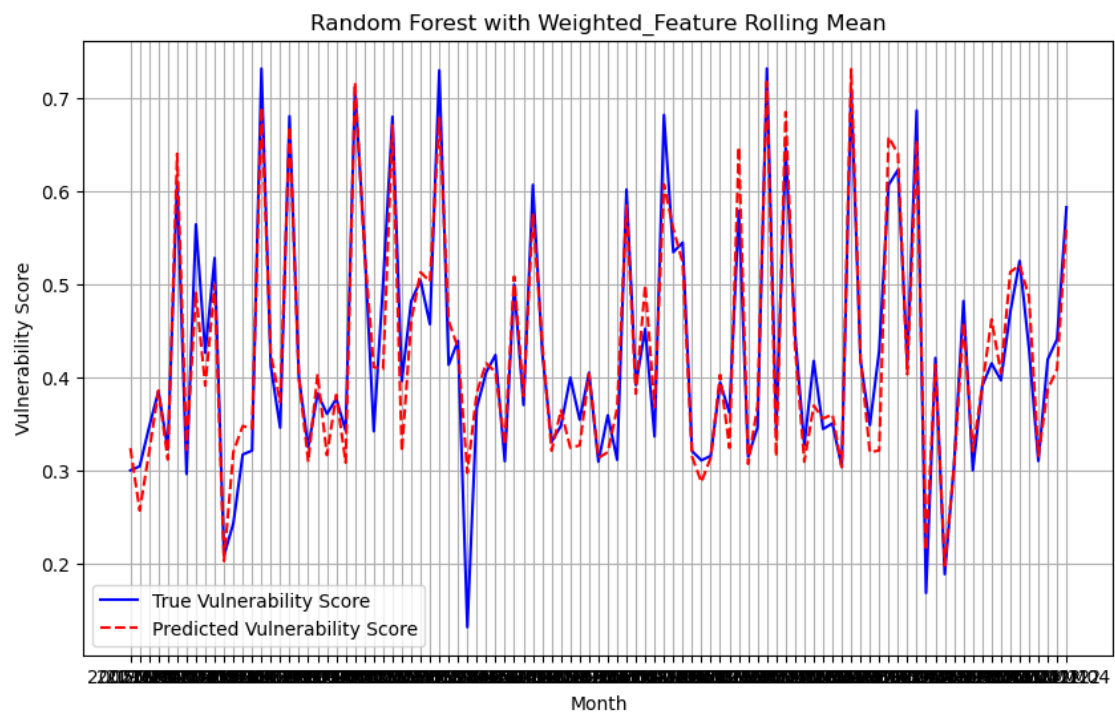
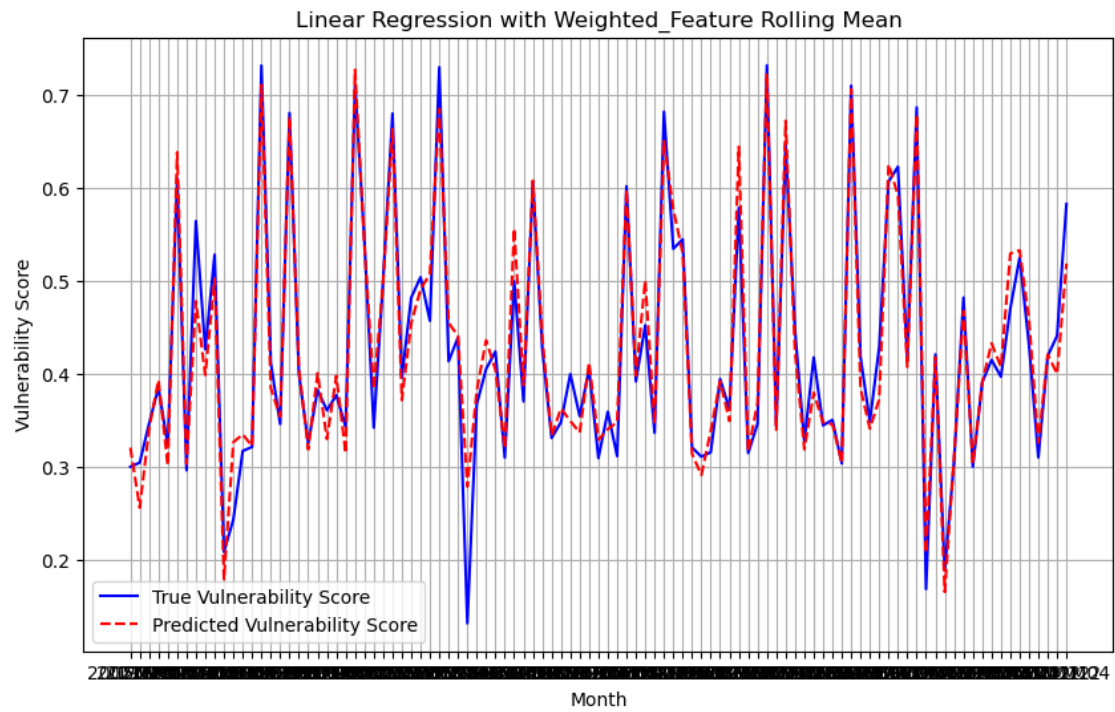
        plt.figure(figsize=(10, 6))
        plt.title(f"{model_name} with {feature} Rolling Mean")
        plt.plot(df_test['Month'], y_test, label='True Vulnerability Score',
↪color='blue')
        plt.plot(df_test['Month'], model_info["predictions"], label='Predicted_
↪Vulnerability Score', color='red', linestyle='--')
        plt.xlabel('Month')
        plt.ylabel('Vulnerability Score')
        plt.legend()
        plt.grid(True)
        plt.show()

```









[]: